

Event-Driven Microservices Architecture for Scalable E-Commerce Platforms: A Study on CDC, Kafka, and Elasticsearch Integration

Nguyen Minh Nhat¹ · Tran Quoc Bao² · Le Thi Lan³

¹ Vietnam Academy of Cryptography Techniques, Hanoi, Vietnam

² Faculty of Information Technology, Hanoi University of Science and Technology

³ School of Computer Science, Vietnam National University

Abstract

Modern e-commerce systems demand high availability, elastic scalability, and near-real-time search capabilities that monolithic architectures cannot economically provide. This paper presents a reference architecture that combines Change Data Capture (CDC) via Debezium, Apache Kafka as a distributed event backbone, and Elasticsearch for full-text product search within a containerised microservices deployment orchestrated by Docker Swarm. We detail the event-sourcing pipeline that propagates PostgreSQL write-ahead log (WAL) mutations to Kafka topics, which are subsequently consumed by an Elasticsearch sink connector to maintain a search index in eventual consistency. A prototype implementation demonstrates sub-200 ms p99 search latency under 500 concurrent users and zero data loss across simulated broker failures. We further analyse trade-offs introduced by eventual consistency and propose idempotency strategies for consumer services. Our findings suggest that the proposed pipeline reduces search-index update lag by 78% compared to scheduled batch synchronisation while adding negligible operational overhead.

Keywords: microservices, event-driven architecture, Debezium CDC, Apache Kafka, Elasticsearch, Docker Swarm, e-commerce

1. Introduction

The rapid growth of online retail has exposed the limitations of tightly coupled, single-database web applications. When a monolith handles product catalogue management, order processing, and search indexing within the same process, a spike in search traffic can degrade checkout performance and vice versa. Microservices architectures address this by decomposing the application into independently deployable units, each owning its data store and communicating through well-defined APIs or asynchronous messaging.

A persistent challenge is keeping derived data stores — such as search indexes or read-model caches — consistent with the primary relational database without burdening application code with dual-write logic. Change Data Capture (CDC) solves this by treating the database transaction log as an event stream. Debezium, an open-source CDC platform, tails PostgreSQL's Write-Ahead Log (WAL) and publishes row-level change events to Apache Kafka topics.

This paper makes the following contributions: (i) a concrete reference architecture for CDC-based search synchronisation in a microservices e-commerce platform; (ii) an empirical evaluation of end-to-end latency and fault tolerance under realistic load; (iii) a discussion of consistency guarantees and idempotent consumer design.

2. Background and Related Work

2.1 Change Data Capture

CDC is a well-established pattern for propagating database mutations to downstream systems without polling. Log-based CDC, as implemented by Debezium, offers lower overhead and higher fidelity than trigger- or query-based approaches because it reads the database's native replication log rather than instrumenting application queries. Debezium connectors wrap each row mutation in an envelope schema containing the before and after images of the record, together with metadata such as the source LSN (Log Sequence Number) and transaction timestamp.

2.2 Apache Kafka as Event Backbone

Apache Kafka provides a partitioned, replicated, durable log that decouples event producers from consumers. Its topic-partition model enables horizontal scaling of both ingestion and consumption, while configurable retention policies allow re-processing of historical events. Kafka Connect, the integration framework bundled with Kafka, hosts Debezium source connectors and Elasticsearch sink connectors in a unified runtime.

2.3 Elasticsearch for Full-Text Search

Elasticsearch is a distributed search and analytics engine built on Apache Lucene. Its inverted-index data structure delivers sub-millisecond full-text queries at scale. In e-commerce, it powers features such as product name autocomplete, faceted navigation by category or price range, and relevance-ranked search results. Keeping the Elasticsearch index consistent with the PostgreSQL product catalogue is the central synchronisation problem addressed in this paper.

3. Proposed Architecture

Figure 1 illustrates the data flow. The product-service writes product records to PostgreSQL. Debezium's PostgreSQL connector, running inside Kafka Connect, consumes the WAL and publishes INSERT, UPDATE, and DELETE events to the Kafka topic `product.public.products`. The Elasticsearch Kafka Connector subscribes to this topic and upserts (or deletes) documents in the products search index. Kong API Gateway routes incoming HTTP requests to the appropriate microservice — search queries go to the search-service, which queries Elasticsearch, while write operations go to the product-service.

Layer	Component	Technology
Ingress	API Gateway	Kong Gateway
Application	Product Service	NestJS + PostgreSQL

Messaging	Event Broker	Apache Kafka + Kafka Connect
CDC	Change Capture	Debezium PostgreSQL Connector
Search	Index Store	Elasticsearch 8.x
Container Orchestration	Cluster	Docker Swarm

Table 1. Technology stack per architectural layer.

3.1 Debezium Configuration

PostgreSQL must be configured with `wal_level = logical` and the `pgoutput` logical decoding plugin enabled. The Debezium connector is registered via the Kafka Connect REST API with `slot.name`, `publication.name`, and transforms settings that flatten the envelope and route each table to a dedicated Kafka topic. Heartbeat topics are configured to advance the replication slot LSN during periods of low write activity, preventing WAL bloat on the primary.

3.2 Idempotent Elasticsearch Sink

The Elasticsearch sink connector uses the Debezium record key (primary key) as the document `_id`. Because Kafka guarantees at-least-once delivery, duplicate events may arrive after a connector restart. Mapping the primary key to `_id` ensures that re-indexing an existing document is idempotent — Elasticsearch performs an upsert rather than creating a duplicate entry.

4. Experimental Evaluation

We deployed the architecture on a three-node Docker Swarm cluster (AWS EC2 t3.medium instances, us-east-1 region). Load was generated with k6, ramping from 50 to 500 virtual users over 10 minutes. Prometheus and Grafana collected metrics from Kafka, Debezium, Elasticsearch, and the application services.

Metric	Value
p50 search latency	34 ms
p99 search latency	187 ms
CDC event lag (median)	41 ms
Throughput (write)	1 240 events/s
Elasticsearch index lag	≤ 120 ms
Data loss events (broker failover)	0

Table 2. Performance results at 500 concurrent users.

The 78% reduction in average index update lag compared to the baseline batch-sync approach (scheduled every 30 seconds) demonstrates the practical advantage of log-based CDC for near-real-time search experiences. During a simulated Kafka broker failure (killing one of three brokers mid-test), the Debezium connector automatically resumed from the last committed offset, and no

product update was lost.

5. Discussion

Eventual consistency introduces a time window during which the Elasticsearch index may not reflect the latest PostgreSQL state. In practice, the measured lag of under 120 ms is imperceptible to end users, but applications requiring strict read-your-writes semantics (e.g., immediately searching for a product just added by the same user session) should route the first post-write search to the product-service rather than Elasticsearch.

Operational complexity increases with the number of connectors and Kafka topics. Schema evolution requires careful use of Confluent Schema Registry or compatible Avro schema changes to avoid breaking downstream consumers. We recommend a schema compatibility policy of `BACKWARD_TRANSITIVE` for all Debezium topics.

6. Conclusion

We presented and empirically evaluated an event-driven microservices architecture for e-commerce platforms that leverages Debezium CDC, Apache Kafka, and Elasticsearch to achieve near-real-time search index synchronisation. The prototype achieved p99 search latency under 200 ms at 500 concurrent users and demonstrated zero data loss across broker failures. Future work will explore integrating a vector search index for semantic product recommendations and extending the architecture to multi-region Active-Active deployments.

References

- [1] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [2] Debezium Authors (2023). Debezium 2.4 Reference Documentation.
<https://debezium.io/documentation/reference/2.4/>
- [3] Kafka Project Management Committee (2023). Apache Kafka Documentation 3.6.
<https://kafka.apache.org/documentation/>
- [4] Elastic N.V. (2024). Elasticsearch Guide 8.12. <https://www.elastic.co/guide/en/elasticsearch/reference/current/>
- [5] Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- [6] Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.
- [7] Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proc. NetDB Workshop*.